

MineCreator

Stage 1 技术实施简要说明

版本 0.1 | 2026-10-04 | Windows / Minecraft Java Edition

目标：用户输入自然语言，LLM 通过 API 产生结构化建筑操作；程序从后台创建、修改指定区域，并读取游戏世界验证结果。

完成示例：“在测试区域建一面石砖墙”，随后“只把最上面一层换成橡木”。两次操作均能在官方 Java 客户端中看到，且读回数据与计划一致。

管线与职责

组件	职责
MineCreator + LLM API	我们编写轻量对话与工具调用流程；模型提出操作，程序校验并执行。
MCPQ Python 客户端 / 插件	通过 gRPC 将程序连接到 Paper，提供方块及区域读写接口。[2][3]
本地 Paper + 官方 Java 客户端	Paper 运行世界；客户端连接本机服务器并显示变化。Paper 是服务端软件。 [1]

数据流：用户输入 → MineCreator（调用 LLM API）→ 校验后的操作 → MCPQ → Paper → 读回检查。

所需资源

资源	准备内容
Minecraft / Paper / Java	官方正版客户端；Paper JAR；匹配服务端版本的 Java 运行时。
Python / MCPQ	Python 3.10+、虚拟环境、mcpq 包与兼容的 Paper 版 MCPQ 插件。[2][4]
模型 API	支持工具调用或结构化输出的模型；API Key、接口地址、模型名及少量测试额度。供应商待选。
测试数据 / GitHub	独立测试世界、限定施工区域；现有 GitHub 仓库保存代码、模板和文档。

范围：先做 CLI、方块与小区域操作。本阶段暂不包含完整 GUI、概念图、档案库或大型建筑；“区域修改”指方块编辑，不涉及重写 Chunk 地形生成器。

环境与基础连接

先验证程序能够修改世界，再接入模型。每一步通过后再继续。

01 确认并记录兼容版本

核对 Minecraft 客户端、Paper、MCPQ 插件与 Python 包的支持范围，固定版本并记录下载来源。Paper 1.20 至 1.21.11 对应 Java 21；26.1+ 对应 Java 25。[1] 优先验证明确支持的版本组合，不把插件文档的“1.20.1+”视为全部未来版本的保证。[2]

通过标准：java -version 与 Python 版本可确认；选定的客户端版本与 Paper 一致。

02 启动本地 Paper 测试世界

在 runtime/server/ 放入下载的 Paper JAR，首次启动生成配置；由用户阅读并同意 Minecraft EULA 后，手动设置 eula=true。保持正版登录验证，创建独立测试世界；运行服务端，再从客户端“多人游戏”连接 localhost:25565。

通过标准：控制台启动完成；官方 Java 客户端能进入世界。测试世界尚未承载重要存档。

```
java -Xms2G -Xmx4G -jar paper.jar --nogui
```

命令为示例：在 runtime/server/ 执行，JAR 名称与内存分配按下载文件和本机资源调整。

03 安装并确认 MCPQ 接口

停服后将兼容的 MCPQ Paper 插件放入 plugins/，重启并检查加载日志。确认 plugins/mcpq/config.yml：接口仅监听 localhost，默认端口 1789；不要开放公网。为 MineCreator 建立 .venv 并安装固定版本的 mcpq。[2][4]

通过标准：Python 客户端能连接接口并读取指定维度中的已知方块；25565 是游戏端口，1789 是程序接口端口。

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install mcpq==<verified-version>
```

安装命令中的 verified-version 必须替换为联调确认的实际版本；最终写入依赖清单。

04 完成无模型的读写冒烟验证

在玩家附近选一块已加载、清晰标记的测试区域。先读取并保存原方块状态，再用 getBlockWithData / setBlock 创建一个方块；用 setBlockCube 建一小段墙，随后替换顶层。读回比较，并用原状态验证恢复。[3]

通过标准：创建、修改和恢复均可见；维度、坐标、方块类型与状态一致。此时不消耗模型 token。

LLM API 与执行闭环

模型提出结构化操作，施工由程序完成；用户确认与执行结果都有记录。

05 先独立验证模型 API

选定供应商及模型，使用其官方 SDK 或接口。将 MODEL_API_KEY、MODEL_BASE_URL、MODEL_NAME 放入本地环境变量或 .env，提交无密钥的 .env.example。先做一次简单请求，再验证工具调用或符合 schema 的结构化输出。

通过标准：API 请求成功；程序能解析响应；鉴权、额度、模型名和超时错误有明确提示。

第一版的工具接口：以下为我们拟定义的包装层

工具	主要输入 / 行为
inspect_region	dimension、两个角点；读取限定区域的方块与状态，返回必要摘要。
set_block / fill_region	维度、坐标、block_id 与 block states；放置方块或填充小区域。
verify_edit	edit_id；读回受影响的位置，比较预期与实际，返回差异。
restore_edit	edit_id；恢复本次记录的原状态。首版仅保证已测试的简单静态方块。

06 实现校验、计划确认与确定性执行

把模型输出解析为操作列表；校验维度、整数坐标、区域边界、材料白名单及修改量。先展示坐标、材料与预计方块数，经用户确认后保存旧状态并批量执行。模型不能直接运行任意 Python、Shell 或服务命令。

通过标准：合法计划能执行；越界、非法材料、超额和格式错误均在修改世界之前拒绝。

起始限制建议：单次最多影响 512 个唯一位置、限定一个测试区域、单轮最多 6 次模型请求；数值均为初始工程设置，可在联调后调整。LLM API 调用不阻塞游戏主线程。

07 闭环检查、日志与重复执行处理

保存 edit_id、设计版本、请求摘要、操作、原状态、预期状态及读回结果。执行后调用 verify_edit，将摘要反馈给模型。超时先查询实际状态，确认差异再修复，避免整批盲目重放；错误重试有次数和费用上限。

通过标准：LLM 的完成答复由读回证据支持；重复 edit_id 不会再次产生同一施工任务。

自然语言验收场景

第一次：“在已标记测试区创建一面 5 格宽、3 格高的石棉墙。”第二次：“只把最上面一层换成橡木板，其余保持不变。”程序应验证 15 个目标位置，其中 5 个在第二次变化，另 10 个保持石棉。

MCPQ 是 Minecraft Protobuf Queries，通过 gRPC 通信；它与 Model Context Protocol（MCP）不同。第一版可直接调用 MCPQ，无需另加 MCP 或完整 agent 平台。[2][4]

注意事项与验收清单

先完成一个可验证的小闭环，再扩大建筑规模。

实施时必须处理的细节

坐标与状态：明确维度和建筑原点，约定 x/y/z 方向；两个角点均包含在 setBlockCube 的范围内，尺寸计算要加 1。验证楼梯朝向、上下半砖等状态时，使用带数据的读取接口。[3]

区块与物理：先在已加载区域测试，再专门验证跨 Chunk 或未加载区域。液体、重力方块、红石、门和容器会引入更新或额外数据；第一版采用简单静态材料，不能承诺完整世界级撤销。

权限与费用：本地接口只供本机调用；API Key 不进入 Git 或日志。批量施工本身不消耗 LLM token；记录每轮模型用量，只回传有限区域摘要，不把完整世界塞进上下文。

持久化与备份：代码、锁文件、配置模板、文档提交并推送 GitHub。runtime/、data/ 和世界存档另做备份；停服后复制所有维度的完整世界目录作为一致快照。服务器用 stop 正常退出，重启后确认修改仍在。

常见失败：按层定位

现象	优先检查
客户端无法进入世界	Paper 是否启动、客户端版本、游戏端口、正版登录与服务端日志。
Python 接口拒绝连接	MCPQ 是否成功加载、host/port、插件与客户端协议兼容性。
修改位置或结果不对	世界维度、原点和角点、区块加载、方块状态及物理更新。
LLM 输出无效 / API 超时	模型工具能力、schema、鉴权与额度；超时后先读回，再决定是否重试。

Stage 1 完成标准

- 官方 Java 客户端能进入本地 Paper 世界；Python 可读写并读回验证。
- 两次自然语言指令完成“创建墙 → 局部修改”，结果与预期一致。
- 非法或越界操作被拒绝；测试范围内的原状态能够恢复。
- 停服重启后修改保留；版本配置与代码已推送 GitHub，运行日志另存本地。

资料与状态

[1] Paper：运行环境、下载与启动

[2] MCPQ 插件：安装、版本与端口配置

[3] MCPQ World API：读写、状态与区域接口

[4] MCPQ Python：安装及客户端兼容说明

资料核对日期：2026-10-04。本文是待实施方案；Paper/MCPQ 尚未在本机联调，最终版本与模型供应商待确认。命令为操作示例，验收标准不代表功能已完成。